Devoir 5 : Réseau de Hopfield (**Facultatif**)

PHQ404

Date de remise : -

1 Objectif

L'objectif de ce devoir est d'implémenter une **mémoire associative** à l'aide d'un réseau de Hopfield. Vous apprendrez à :

- Stocker des motifs dans le réseau,
- Rappeler ces motifs à partir d'entrées bruitées ou incomplètes,
- Observer la dynamique du réseau et comprendre la convergence vers des états stables.

Ce travail illustre un mécanisme simple de correction d'erreur et introduit des concepts fondamentaux de neuroscience computationnelle et de physique statistique appliquée aux réseaux de neurones.

2 Introduction

John Hopfield (1933-) est un physicien et biologiste théorique américain dont les contributions ont profondément marqué le domaine de l'intelligence artificielle et des neurosciences computationnelles. Il est surtout connu pour avoir introduit en 1982 le **réseau de Hopfield**, un type de réseau de neurones artificiel possédant une capacité de mémoire associative.

L'idée centrale est la suivante : le réseau stocke des motifs sous forme d'**états stables**. Lorsqu'on lui fournit une version incomplète ou bruitée d'un motif, sa dynamique interne le fait évoluer vers l'état stable correspondant à un souvenir stocké.

Idée clé

Un réseau de Hopfield fonctionne comme un système de récupération d'information avec correction d'erreur : chaque motif mémorisé correspond à un **minimum local** d'une fonction d'énergie, et la dynamique du réseau fait converger toute entrée proche vers ce minimum.

Cette approche a inspiré de nombreuses recherches sur les algorithmes d'optimisation et les modèles de mémoire du cerveau, et a permis d'établir un pont entre physique et biologie computationnelle.

Articles fondateurs :

— J. J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities.," *Proceedings of the national academy of sciences*, t. 79, n° 8, p. 2554-2558, 1982 [1]
— J. J. Hopfield, "Neurons with graded response have collective computational properties like those of two-state neurons.," *Proceedings of the national academy of sciences*, t. 81, n° 10, p. 3088-3092, 1984 [2]

2.1 Définition du réseau de Hopfield

Un réseau de Hopfield est un réseau de neurones de type McCulloch-Pitts avec les caractéristiques suivantes :

— **Réseau complet et simple** : chaque neurone est connecté à tous les autres neurones, sauf à lui-même. Le réseau est donc récurrent et sans boucle.
— **Poids symétriques** : $W = W^\top$, donc $w_{ij} = w_{ji}$ pour tous les neurones i, j.
— **Poids positifs ou négatifs** : permet de modéliser l'**inhibition**, où l'activité d'un neurone peut réduire celle d'un autre, limitant ainsi la transmission de signaux excitateurs.
— **Plasticité selon Hebb** : les connexions se renforcent lorsque les neurones sont activés simultanément. Formulé par **Donald Hebb (1949)** : *"neurons that fire together, wire together"*.

Remarque

Cette structure permet au réseau de représenter les souvenirs de manière **distribuée** : chaque neurone participe à plusieurs motifs, et chaque motif est représenté par plusieurs neurones.

2.2 Dynamique du réseau

La dynamique du réseau de Hopfield est définie comme suit :

— **Activité binaire** : l'état du neurone i à l'instant t est $x_i(t) \in \{0, 1\}$.
— **Temps discret** : $t \in \{0, 1, 2, \dots\}$.
— **Mise à jour asynchrone** : à chaque pas de temps, un seul neurone est choisi aléatoirement pour mise à jour.
— **Règle de mise à jour** : Si le neurone i à l'instant t est mis à jour, alors

$$x_i(t+1) = H\left(\sum_{j=1}^{n} w_{ij} x_j(t) - \theta_i\right),$$

où H est la fonction de Heaviside. Sous forme vectorielle :

$$\mathbf{x}(t+1) = H(W\mathbf{x}(t) - \boldsymbol{\theta}).$$

2.3 Concepts fondamentaux

Voici quelques concepts fondamentaux nécessaires pour discuter du modèle de Hopfield :

— **État stationnaire** : $\mathbf{x}^*$ tel que $\mathbf{x}(t) = \mathbf{x}^*$ pour tout t.
— **Fonction d'énergie** (fonction de Lyapunov) :

$$E(\mathbf{x}) = -\frac{1}{2}\sum_{i\neq j} w_{ij}x_ix_j + \sum_i \theta_i x_i = -\frac{1}{2}\mathbf{x}^\top W\mathbf{x} + \boldsymbol{\theta}^\top\mathbf{x}.$$

— **Résultat fondamental de Hopfield** : avec mise à jour asynchrone et poids symétriques,

$$E(\mathbf{x}(t+1)) \leq E(\mathbf{x}(t)),$$

ce qui garantit que les minima locaux de E correspondent à des états stationnaires stables.
— **Mémoire** : chaque état stationnaire stable représente un motif mémorisé. À partir d'un état proche, le réseau converge vers ce motif.

2.4 Plasticité et apprentissage d'états

— **Apprentissage d'un état** : pour stocker un seul état $\mathbf{y}$, on change $W_{i,j}$ pour

$$W_{ij} \leftarrow W_{ij} + \begin{cases} (2y_i - 1)(2y_j - 1), & i \neq j \\ 0, & i = j \end{cases}$$

— **Apprentissage de plusieurs états** : pour stocker les $\mathbf{y}^\mu$, $\mu = 1, \ldots, p$, on change $W_{i,j}$ pour

$$W_{ij} \leftarrow W_{ij} + \begin{cases} \sum_\mu (2y_i^\mu - 1)(2y_j^\mu - 1), & i \neq j \\ 0, & i = j \end{cases}$$

— **Lien avec Hebb** : les termes $y_i^\mu y_j^\mu$ renforcent les connexions lorsque les neurones i et j sont activés simultanément.

Remarque

Stocker trop d'états peut créer des **états parasites** (minima locaux non désirés), ce qui réduit la fiabilité du rappel.

3 Énoncé

3.1 Implémentation du réseau

Vous allez devoir implémenter une classe `HopfieldNetwork` qui créée un réseau de Hopfield. Dans celle-ci, vous devrez implémenter les méthodes suivantes :
— `__init__` : le constructeur de la classe qui prend en argument les poids de connexions `weights`, les seuils `thresholds` et l'état initial `initial_state` du réseau.
— `n` : Une propriété qui retourne le nombre de neurones dans le réseau.
— `energy` : Une méthode qui retourne l'énergie du réseau.
— `get_state_energy` : Une méthode qui calcule l'énergie d'un état donné.

— `set_state` : Une méthode qui permet de changer l'état du réseau.
— `update` : Une méthode qui met à jour l'état du réseau de manière asynchrone sur un pas de temps.
— `simulate` : Une méthode qui met à jour l'état du réseau de manière asynchrone sur un nombre de pas de temps `m` et qui retourne l'historique des états du réseau.
— `learn_one_state` : Une méthode qui permet d'apprendre un état `state` en modifiant les poids de connexions.
— `learn_many_states` : Une méthode qui permet d'apprendre plusieurs états `states` en modifiant les poids de connexions.

3.2 Visualisation des résultats

Vous devez produire :
— Graphiques de l'énergie du réseau en fonction du temps pour différents motifs mémorisés.
— Visualisation de l'évolution de l'état du réseau en fonction du temps (GIFs recommandés).

Les tests doivent inclure les états `image` et `message` dans `tests/data`. Ajoutez une courte analyse sur l'apprentissage de plusieurs états et sur la dynamique du réseau.

4 Barème d'évaluation

Ce devoir est **facultatif**. 5% en points bonus est attribué pour sa complétion (en comparaison avec 10% pour les devoirs précédents). 3% est attribué pour le code, et 2% est attribué pour le rapport. Les critères d'évaluation pour le code restent les mêmes que pour le devoir 4. Les exigences pour le rapport sont légèrement différentes : seules les sections *Résumé*, *Résultats*, *Discussion* sont demandées.

Références

[1] J. J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities.," *Proceedings of the national academy of sciences*, t. 79, n° 8, p. 2554-2558, 1982.

[2] J. J. Hopfield, "Neurons with graded response have collective computational properties like those of two-state neurons.," *Proceedings of the national academy of sciences*, t. 81, n° 10, p. 3088-3092, 1984.